

Formative Assignment: Advanced Linear Algebra (PCA)

This notebook will guide you through the implementation of Principal Component Analysis (PCA). Fill in the missing code and provide the required answers in the appropriate sections. You will work with a dataset that is Africanized .

1. Make sure to display outputs for each code cell when submitting.
2. Do not write all your code on one cell
3. Do not use any libraries aside from numpy

Group: PCA Pair Team 15: Teta Butera Nelly and Mutoni Keira

GitHub repo: github.com/nellybutera/pca-african-energy-systems

Topic: What drives differences in energy systems across 54 African countries, 2000-2022?

Dataset: [Our World in Data energy dataset](#) (`owid-energy-data.csv`), filtered to the 54 African countries, 2000-2022. The raw file has real blank cells (GDP, oil, gas and coal production). It has text columns (`country` , `iso_code`). We add one more, `sub_region` , from the UN geoscheme.

Libraries: only `numpy` and `matplotlib`. The standard-library modules `os` , `urllib` and `time` are used only to download the file and time the benchmarks. No `pandas` , `sklearn` or `scipy` .

Part A. Data handling

Steps: load the raw file, keep African rows, find the missing values, impute them, encode the text columns, and fix skewed columns.

```
import os, time, urllib.request
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams["figure.dpi"] = 110
DATA_PATH = "data/owid-energy-data.csv"
# pinned to the exact OWID commit we used, so a re-run in Colab gives the same numbers
URL = "https://raw.githubusercontent.com/owid/energy-data/7e387a16f70a510e433f8aac7efeac6faa1e5059/owid-energy-data.csv"
if not os.path.exists(DATA_PATH):          # Colab: fetch the raw file if the repo copy is not mounted
    os.makedirs("data", exist_ok=True)
    urllib.request.urlretrieve(URL, DATA_PATH)

FEATURES = ["gdp", "population", "primary_energy_consumption", "energy_per_capita", "electricity_demand",
            "per_capita_electricity", "fossil_share_elec", "renewables_share_elec", "hydro_share_elec",
            "solar_share_elec", "oil_production", "gas_production", "coal_production",
            "greenhouse_gas_emissions", "net_elec_imports_share_demand"]
TEXT_COLS = ["country", "iso_code"]

# read only the columns we need (the raw file has 130 columns)
header = open(DATA_PATH, encoding="utf-8").readline().strip().split(",")
wanted = TEXT_COLS + ["year"] + FEATURES
raw = np.loadtxt(DATA_PATH, dtype=str, delimiter=",", comments=None, quotechar="'", encoding="utf-8",
                 skiprows=1, usecols=[header.index(c) for c in wanted])
```

```
print("Raw rows x selected columns:", raw.shape, "(raw file has", len(header), "columns)")
raw[:3]
```

Raw rows x selected columns: (23377, 18) (raw file has 130 columns)

```
array([[ 'ASEAN (Ember)', '', '2000', '', '', '', '', '378.760', '', '80.666', '19.334', '13.314', '0.000',
        '', '', '', '216.870', ''],
       [ 'ASEAN (Ember)', '', '2001', '', '', '', '', '405.090', '', '80.945', '19.055', '13.412', '0.000',
        '', '', '', '231.010', ''],
       [ 'ASEAN (Ember)', '', '2002', '', '', '', '', '433.190', '', '82.333', '17.667', '12.304', '0.000',
        '', '', '', '248.080', '']], dtype='<U37')
```

```
AFRICA = ("DZA AGO BEN BWA BFA BDI CPV CMR CAF TCD COM COD COG CIV DJI EGY GNQ ERI SWZ ETH GAB GMB GHA GIN GNB KEN
"
         "LSO LBR LBY MDG MWI MLI MRT MUS MAR MOZ NAM NER NG A RWA STP SEN SYC SLE SOM ZAF SSD SDN TZA TGO TUN UGA
ZMB ZWE").split()
SUBREGION = {}
for r, codes in {
    "North":    "DZA EGY LBY MAR SDN TUN",
    "West":     "BEN BFA CPV CIV GMB GHA GIN GNB LBR MLI MRT NER NG A SEN SLE TGO",
    "East":     "BDI COM DJI ERI ETH KEN MDG MWI MUS MOZ RWA SYC SOM SSD TZA UGA ZMB ZWE",
    "Central":  "AGO CMR CAF TCD COD COG GNQ GAB STP",
    "Southern": "BWA SWZ LSO NAM ZAF"}.items():
    for c in codes.split():
        SUBREGION[c] = r
assert set(SUBREGION) == set(AFRICA) and len(AFRICA) == 54

year_all = raw[:, 2].astype(int)
keep = np.isin(raw[:, 1], AFRICA) & (year_all >= 2000) & (year_all <= 2022)
rows = raw[keep]
country, iso, year = rows[:, 0], rows[:, 1], rows[:, 2].astype(int)
region = np.array([SUBREGION[c] for c in iso]) # new text column added by us
X_raw = np.where(rows[:, 3:] == "", "nan", rows[:, 3:]).astype(float) # blank cell -> NaN
print("African country-years:", X_raw.shape[0], "| countries:", len(np.unique(country)), "| numeric features:",
X_raw.shape[1])
```

African country-years: 1230 | countries: 54 | numeric features: 15

```
# Which cells are actually missing in the raw data?
n = X_raw.shape[0]
miss_count = np.isnan(X_raw).sum(axis=0)
print(f"{'feature':34s}{ 'missing':>8s}{ '%':>8s}")
for f, m in zip(FEATURES, miss_count):
    print(f"{'f':34s}{m:8d}{100*m/n:8.1f}")
print("\nTotal missing cells:", int(miss_count.sum()), "of", X_raw.size, f"
({100*miss_count.sum()/X_raw.size:.1f}%)")

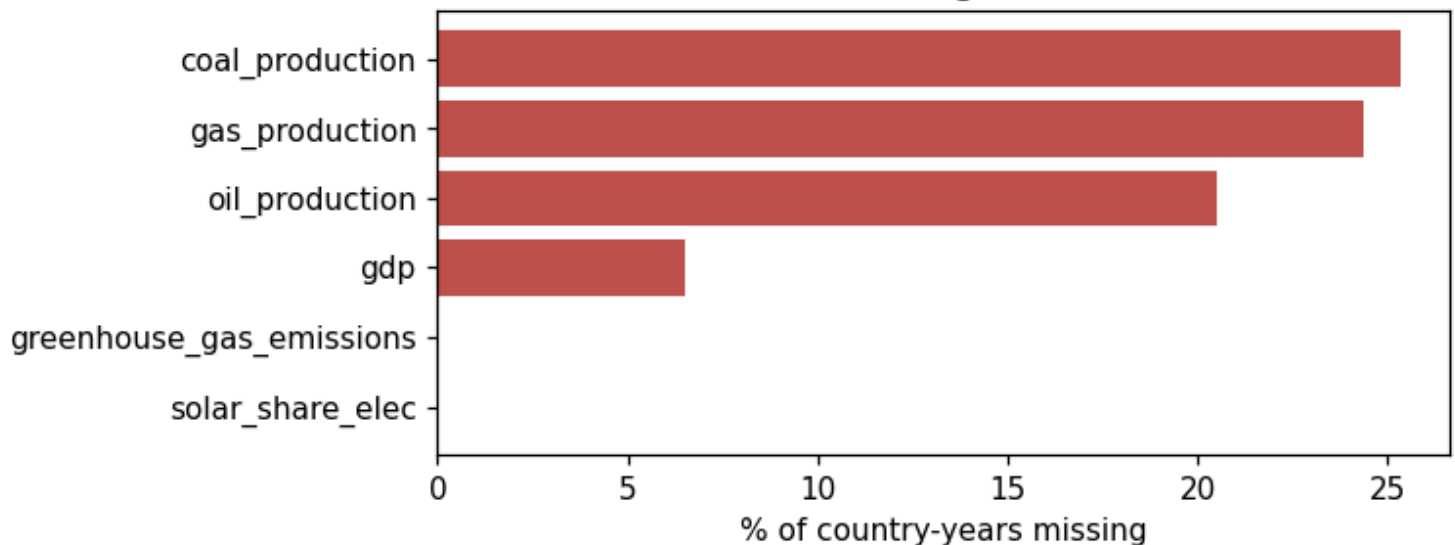
order = np.argsort(miss_count)[::-1][:6]
plt.figure(figsize=(7, 3))
plt.barh([FEATURES[i] for i in order][::-1], (100 * miss_count[order] / n)[::-1], color="#c0504d")
plt.xlabel("% of country-years missing"); plt.title("Columns with missing values (raw data)")
plt.tight_layout(); plt.show()
```

feature	missing	%
gdp	80	6.5
population	0	0.0

primary_energy_consumption	0	0.0
energy_per_capita	0	0.0
electricity_demand	0	0.0
per_capita_electricity	0	0.0
fossil_share_elec	0	0.0
renewables_share_elec	0	0.0
hydro_share_elec	0	0.0
solar_share_elec	0	0.0
oil_production	252	20.5
gas_production	300	24.4
coal_production	312	25.4
greenhouse_gas_emissions	0	0.0
net_elec_imports_share_demand	0	0.0

Total missing cells: 944 of 18450 (5.1%)

Columns with missing values (raw data)



```
# Which columns are NOT numeric?
for name, arr in [("country", country), ("iso_code", iso), ("sub_region (added)", region)]:
    print(f"{name:20s} text column, {len(np.unique(arr))} distinct values, e.g. {np.unique(arr)[:4].tolist()}")
```

country	text column, 54 distinct values, e.g. ['Algeria', 'Angola', 'Benin', 'Botswana']
iso_code	text column, 54 distinct values, e.g. ['AGO', 'BDI', 'BEN', 'BFA']
sub_region (added)	text column, 5 distinct values, e.g. ['Central', 'East', 'North', 'Southern']

Why this imputation. The blank oil, gas and coal cells sit inside a country's own time series (each of those countries has other years with values), so we fill them from that country's own years: linear in time, and the nearest observed value at the ends. A country's production changes slowly, so this is closer to the truth than one continent-wide average. GDP is different. Eritrea, Somalia, South Sudan and Sudan have no GDP value in any year, so there is nothing to interpolate from. For those four we use the median GDP of their sub-region. This is the weakest step in the notebook: it is a rough estimate and hides real differences inside a sub-region (Sudan is placed with the wealthier North African economies, for example).

```
def impute_column(col, countries, regions, years):
    """Stage 1: fill a country's gaps from that country's own other years (linear in time, nearest value at the ends).
    Stage 2: if a country has no data at all in this column, use the median of its sub-region."""
    col = col.copy(); s1 = s2 = 0
    for c in np.unique(countries):
```

```

idx = np.where(countries == c)[0]
idx = idx[np.argsort(years[idx])]
v = col[idx]; ok = ~np.isnan(v)
if ok.any() and not ok.all():
    filled = np.interp(years[idx], years[idx][ok], v[ok])
    col[idx[~ok]] = filled[~ok]; s1 += int((~ok).sum())
for r in np.unique(regions):
    m = regions == r; miss = m & np.isnan(col)
    if miss.any():
        med = np.nanmedian(col[m])
        col[miss] = med if not np.isnan(med) else np.nanmedian(col)
        s2 += int(miss.sum())
return col, s1, s2

X_imp = X_raw.copy()
print(f"{'feature':34s}{'own-country':>12s}{'sub-region':>12s}")
for j, f in enumerate(FEATURES):
    if np.isnan(X_raw[:, j]).any():
        X_imp[:, j], s1, s2 = impute_column(X_raw[:, j], country, region, year)
        print(f"{'f':34s}{'s1':12d}{'s2':12d}")
print("\nNaN left after imputation:", int(np.isnan(X_imp).sum()))
no_gdp = [str(u) for u in np.unique(country) if np.isnan(X_raw[country == u, 0]).all()]
print("Countries with NO GDP value in any year (filled from their sub-region median):", no_gdp)

```

feature	own-country	sub-region
gdp	0	80
oil_production	252	0
gas_production	300	0
coal_production	312	0

NaN left after imputation: 0

Countries with NO GDP value in any year (filled from their sub-region median): ['Eritrea', 'Somalia', 'South Sudan', 'Sudan']

```

# Encoding the text columns
# 1) country -> integer id (kept only as a label; a country id has no numeric meaning, so it is NOT a PCA feature)
countries_sorted, country_id = np.unique(country, return_inverse=True)
# 2) sub_region -> one-hot, dropping the first category to avoid perfectly collinear columns
REGIONS = ["Central", "East", "North", "Southern", "West"]
onehot = np.stack([(region == r).astype(float) for r in REGIONS[1:]], axis=1)
ONEHOT_NAMES = [f"region_{r}" for r in REGIONS[1:]]
print("One-hot columns:", ONEHOT_NAMES, "| reference category:", REGIONS[0])
print("Rows per sub-region:", {r: int((region == r).sum()) for r in REGIONS})

```

One-hot columns: ['region_East', 'region_North', 'region_Southern', 'region_West'] | reference category: Central
 Rows per sub-region: {'Central': 207, 'East': 402, 'North': 138, 'Southern': 115, 'West': 368}

```

# Skewed columns: GDP, population and fuel production are dominated by a few giants (Nigeria, Egypt, South Africa...)
def skewness(x):
    z = (x - x.mean(0)) / x.std(0)
    return (z ** 3).mean(0)

sk_before = skewness(X_imp)
to_log = (sk_before > 1) & (X_imp.min(axis=0) >= 0)
X_t = X_imp.copy()
X_t[:, to_log] = np.log1p(X_imp[:, to_log])

```

```

sk_after = skewness(X_t)
print(f"{'feature':34s}{'skew before':>12s}{'skew after':>12s} log1p?")
for j, f in enumerate(FEATURES):
    print(f"{'f':34s}{sk_before[j]:12.2f}{sk_after[j]:12.2f} {'yes' if to_log[j] else ''}")

ALL_FEATURES = FEATURES + ONEHOT_NAMES
data = np.hstack([X_t, onehot]) # final numeric matrix
print("\nFinal matrix:", data.shape, "->", data.shape[0], "country-years x", data.shape[1], "features")

```

feature	skew before	skew after	log1p?
gdp	3.82	0.12	yes
population	3.22	-0.56	yes
primary_energy_consumption	4.32	0.78	yes
energy_per_capita	3.00	0.39	yes
electricity_demand	4.92	1.21	yes
per_capita_electricity	3.00	0.16	yes
fossil_share_elec	-0.43	-0.43	
renewables_share_elec	0.43	0.43	
hydro_share_elec	0.54	0.54	
solar_share_elec	7.75	2.76	yes
oil_production	3.37	1.21	yes
gas_production	4.72	1.94	yes
coal_production	7.12	4.63	yes
greenhouse_gas_emissions	5.49	2.13	yes
net_elec_imports_share_demand	0.79	0.79	

Final matrix: (1230, 19) -> 1230 country-years x 19 features

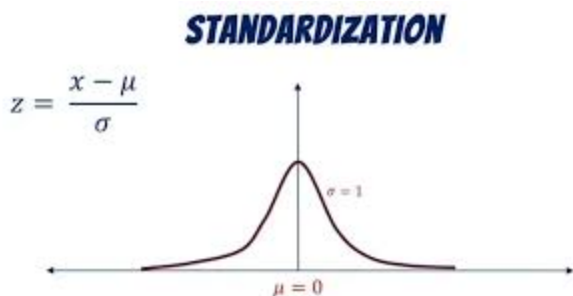
Part B. PCA from scratch (Task 1)

Step 1: Load and Standardize the Data

Before applying PCA, we must standardize the dataset. Standardization ensures that all features have a mean of 0 and a standard deviation of 1, which is essential for PCA.

Fill in the code to standardize the dataset.

STRICTLY - Write code that implements standardization based on the image below



```

# Step 1: Load and Standardize the data (use of numpy only allowed)
mu = np.mean(data, axis=0)
sigma = np.std(data, axis=0)
standardized_data = (data - mu) / sigma
print("column means (should be ~0):", np.round(standardized_data.mean(axis=0)[:5], 10), "...")

```

```
print("column std (should be 1): ", np.round(standardized_data.std(axis=0)[:5], 10), "...")
standardized_data[:5] # Display the first few rows of standardized data
```

```
column means (should be ~0): [-0. -0.  0.  0.  0.] ...
column std (should be 1): [1. 1. 1. 1. 1.] ...
```

```
array([[ 1.3342,  0.8484,  1.7947,  1.2102,  1.4519,  0.9352,  1.0614, -1.0564, -0.9785, -0.4437,  2.24  ,
         3.3453, -0.2966,  1.7408, -0.3865, -0.6968,  2.813  , -0.3212, -0.6534],
       [ 1.3761,  0.8571,  1.8169,  1.2258,  1.4878,  0.9572,  1.0596, -1.0546, -0.9767, -0.4437,  2.2333,
         3.311  , -0.2966,  1.7784, -0.383  , -0.6968,  2.813  , -0.3212, -0.6534],
       [ 1.433  ,  0.8655,  1.8389,  1.2416,  1.5172,  0.9736,  1.0609, -1.0559, -0.9779, -0.4437,  2.2658,
         3.3269, -0.2966,  1.8088, -0.3783, -0.6968,  2.813  , -0.3212, -0.6534],
       [ 1.4992,  0.8739,  1.8674,  1.2647,  1.5686,  1.0093,  1.0432, -1.0382, -0.9598, -0.4437,  2.3126,
         3.3573, -0.2966,  1.8585, -0.3741, -0.6968,  2.813  , -0.3212, -0.6534],
       [ 1.5487,  0.8827,  1.8908,  1.2815,  1.6103,  1.0367,  1.0453, -1.0403, -0.9619, -0.4437,  2.3368,
         3.3464, -0.2966,  1.9041, -0.3742, -0.6968,  2.813  , -0.3212, -0.6534]])
```

Step 3: Calculate the Covariance Matrix

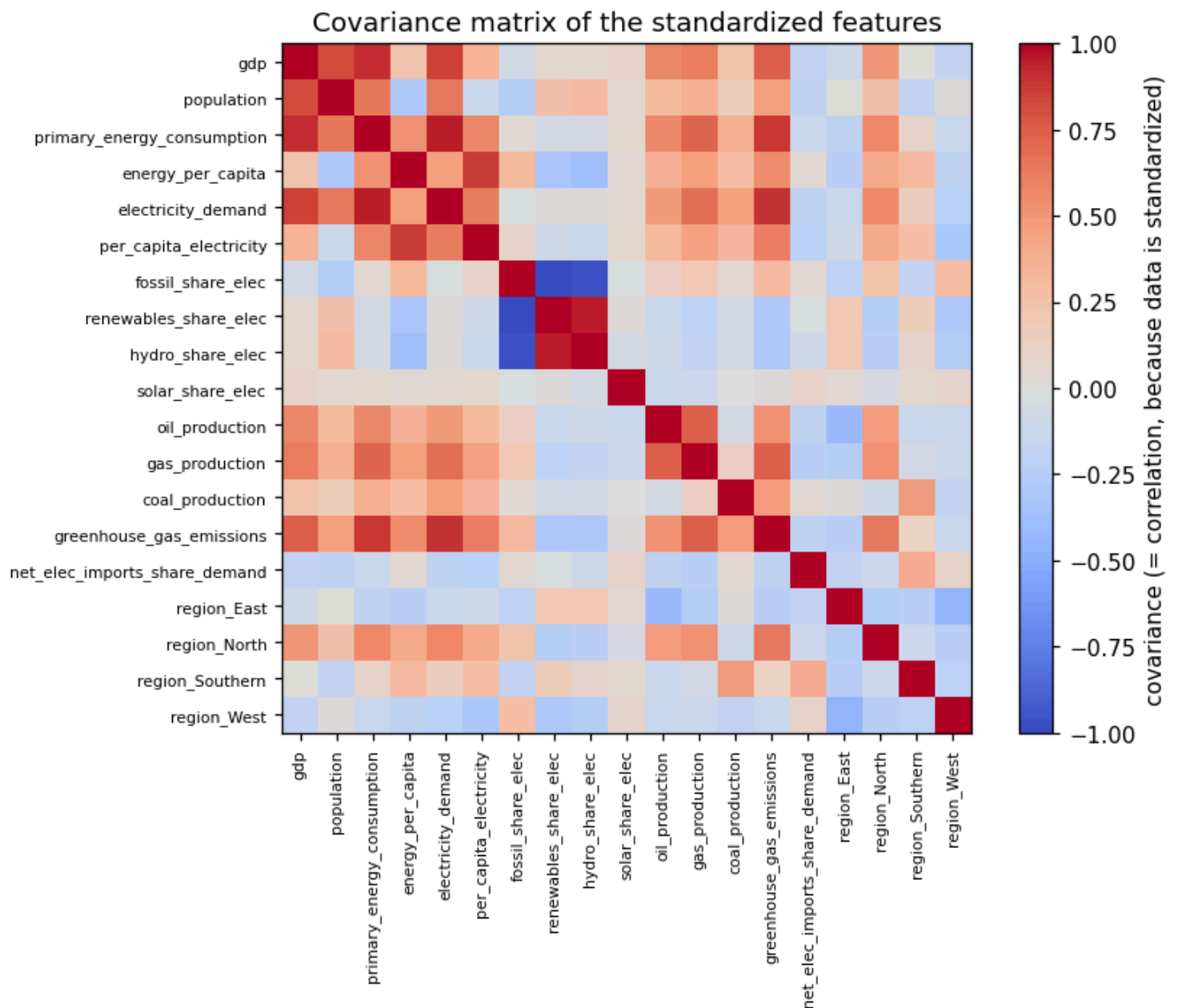
The covariance matrix helps us understand how the features are related to each other. It is a key component in PCA.

```
# Step 3: Calculate the Covariance Matrix
n_obs = standardized_data.shape[0]
cov_matrix = (standardized_data.T @ standardized_data) / n_obs # data is already centred; N matches the
population sigma used in Step 1
print("shape:", cov_matrix.shape, "| matches np.cov:", np.allclose(cov_matrix, np.cov(standardized_data,
rowvar=False, bias=True)))
cov_matrix
```

```
shape: (19, 19) | matches np.cov: True
```

```
array([[ 1.      ,  0.8194,  0.9109, ...,  0.5045,  0.0066, -0.172 ],
       [ 0.8194,  1.      ,  0.6464, ...,  0.259 , -0.1664,  0.017 ],
       [ 0.9109,  0.6464,  1.      , ...,  0.569 ,  0.0892, -0.1448],
       ...,
       [ 0.5045,  0.259 ,  0.569 , ...,  1.      , -0.1142, -0.2323],
       [ 0.0066, -0.1664,  0.0892, ..., -0.1142,  1.      , -0.2098],
       [-0.172 ,  0.017 , -0.1448, ..., -0.2323, -0.2098,  1.      ]], shape=(19, 19))
```

```
plt.figure(figsize=(8, 6.5))
im = plt.imshow(cov_matrix, cmap="coolwarm", vmin=-1, vmax=1)
plt.xticks(range(len(ALL_FEATURES)), ALL_FEATURES, rotation=90, fontsize=7)
plt.yticks(range(len(ALL_FEATURES)), ALL_FEATURES, fontsize=7)
plt.colorbar(im, label="covariance (= correlation, because data is standardized)")
plt.title("Covariance matrix of the standardized features")
plt.tight_layout(); plt.show()
```



In a paragraph that has a maximum of 5 lines, Explain why we need to compute a covariance matrix, provide at least 2 reasons

>  **TO DO – Mutoni Keira:** write your answer here in your own words, **maximum 5 lines, at least 2 reasons**. Hints: what is PCA searching for, and what does the covariance matrix tell us about the features? The heatmap above and the eigenvalue check below may help.

Step 4: Perform Eigendecomposition

Eigendecomposition of the covariance matrix will give us the eigenvalues and eigenvectors, which are essential for PCA. Fill in the code to compute the eigenvalues and eigenvectors of the covariance matrix.

```
# Step 4: Perform Eigendecomposition
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix) # eigh: cov_matrix is symmetric, so eigenvalues are real
print("sum of eigenvalues:", round(eigenvalues.sum(), 6), "(= number of features, because the data is
standardized:", cov_matrix.shape[0], ")")
print("check C v = lambda v:", np.allclose(cov_matrix @ eigenvectors, eigenvectors * eigenvalues))
eigenvalues, eigenvectors
```

```
sum of eigenvalues: 19.0 (= number of features, because the data is standardized: 19 )
check C v = lambda v: True
```

```
(array([0.      , 0.001 , 0.014 , 0.0316, 0.0391, 0.0539, 0.0827, 0.0971, 0.261 , 0.2744, 0.486 , 0.5525,
        0.8893, 1.0372, 1.361 , 1.5825, 2.2324, 3.5279, 6.4764]), array([[ 0.0014, -0.0227,  0.3464, ..., -0.1453,
-0.1941, -0.3241],
        [-0.0032,  0.4958, -0.5056, ..., -0.322 , -0.3009, -0.1891],
        [-0.0016, -0.6818, -0.2328, ..., -0.0136, -0.1062, -0.3716],
        ...,
        [-0.0034, -0.0095,  0.017 , ..., -0.1227,  0.0591, -0.2683],
        [ 0.0012, -0.0067, -0.0156, ...,  0.5364, -0.0462, -0.0361],
        [-0.0005, -0.0182,  0.0706, ..., -0.2526,  0.2014,  0.0754]]), shape=(19, 19)))
```

Step 5: Sort Principal Components

Sort the eigenvectors based on their corresponding eigenvalues in descending order. The higher the eigenvalue, the more important the eigenvector.

Complete the code to sort the eigenvectors and print the sorted components.

[How Is Explained Variance Used In PCA?](#)

```
# Step 5: Sort Principal Components
sorted_indices = np.argsort(eigenvalues)[::-1]          # descending
sorted_eigenvalues = eigenvalues[sorted_indices]
sorted_eigenvectors = eigenvectors[:, sorted_indices]
# eigenvectors are only defined up to sign: fix it so the largest-magnitude loading of each PC is positive
signs = np.sign(sorted_eigenvectors[np.abs(sorted_eigenvectors).argmax(axis=0),
np.arange(sorted_eigenvectors.shape[1])])
sorted_eigenvectors = sorted_eigenvectors * signs
print("eigenvalues in descending order:", np.all(np.diff(sorted_eigenvalues) <= 0))
print("eigenvectors orthonormal:", np.allclose(sorted_eigenvectors.T @ sorted_eigenvectors,
np.eye(len(sorted_eigenvalues))))
sorted_eigenvectors
```

```
eigenvalues in descending order: True
eigenvectors orthonormal: True
```

```
array([[ 0.3241, -0.1941, -0.1453, ...,  0.3464,  0.0227,  0.0014],
        [ 0.1891, -0.3009, -0.322 , ..., -0.5056, -0.4958, -0.0032],
        [ 0.3716, -0.1062, -0.0136, ..., -0.2328,  0.6818, -0.0016],
        ...,
        [ 0.2683,  0.0591, -0.1227, ...,  0.017 ,  0.0095, -0.0034],
        [ 0.0361, -0.0462,  0.5364, ..., -0.0156,  0.0067,  0.0012],
        [-0.0754,  0.2014, -0.2526, ...,  0.0706,  0.0182, -0.0005]], shape=(19, 19))
```

Part C. Choosing the number of components (Task 2)

The number of components is chosen from the explained variance, not by hand.

```
explained_variance_ratio = sorted_eigenvalues / sorted_eigenvalues.sum()
cumulative = np.cumsum(explained_variance_ratio)
```

```
def choose_k(eigvals_desc, threshold=0.90):
    """Smallest k whose components together explain at least `threshold` of the total variance."""
    cum = np.cumsum(eigvals_desc) / eigvals_desc.sum()
    return int(np.searchsorted(cum, threshold) + 1)

print(f'{"PC":>4s}{\'eigenvalue\':>12s}{\'explained %\':>13s}{\'cumulative %\':>14s}')
for i in range(len(sorted_eigenvalues)):
    print(f'{"PC"+str(i+1):>4s}{sorted_eigenvalues[i]:12.3f}{100*explained_variance_ratio[i]:13.2f}{100*cumulative[i]:14.2f}')

print("\nComponents needed for each variance target:")
for t in (0.70, 0.80, 0.85, 0.90, 0.95, 0.99):
    print(f" {int(t*100)}% -> k = {choose_k(sorted_eigenvalues, t)}")
print("Kaiser rule (eigenvalue > 1) keeps k =", int((sorted_eigenvalues > 1).sum()))
```

PC	eigenvalue	explained %	cumulative %
PC1	6.476	34.09	34.09
PC2	3.528	18.57	52.65
PC3	2.232	11.75	64.40
PC4	1.583	8.33	72.73
PC5	1.361	7.16	79.90
PC6	1.037	5.46	85.36
PC7	0.889	4.68	90.04
PC8	0.552	2.91	92.94
PC9	0.486	2.56	95.50
PC10	0.274	1.44	96.95
PC11	0.261	1.37	98.32
PC12	0.097	0.51	98.83
PC13	0.083	0.44	99.27
PC14	0.054	0.28	99.55
PC15	0.039	0.21	99.75
PC16	0.032	0.17	99.92
PC17	0.014	0.07	99.99
PC18	0.001	0.01	100.00
PC19	0.000	0.00	100.00

Components needed for each variance target:

```
70% -> k = 4
80% -> k = 6
85% -> k = 6
90% -> k = 7
95% -> k = 9
99% -> k = 13
```

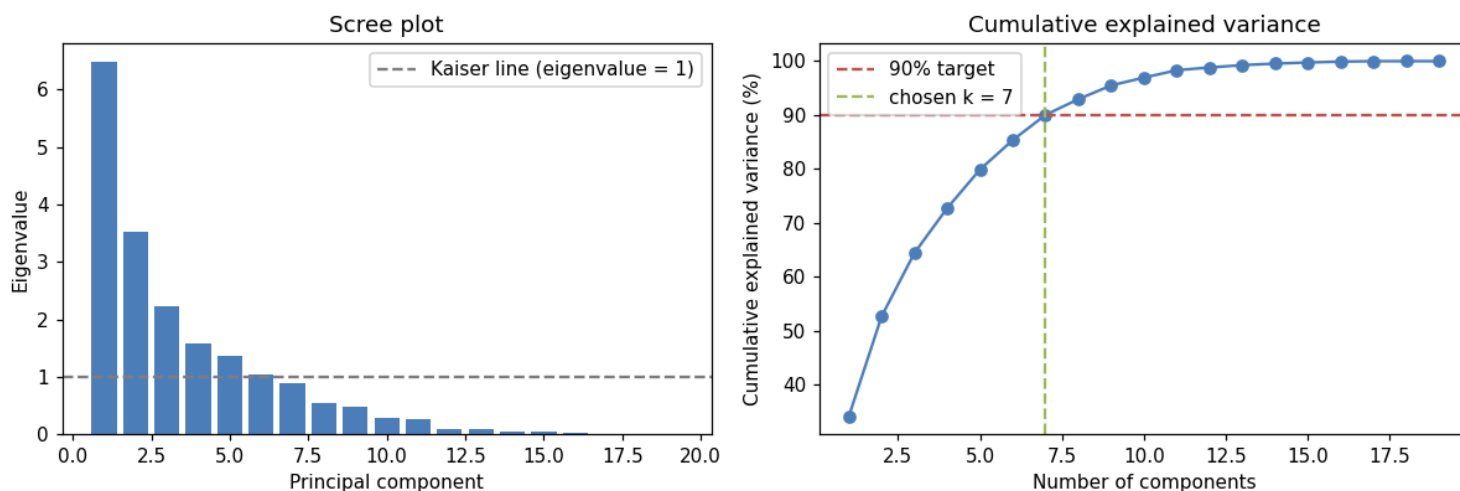
Kaiser rule (eigenvalue > 1) keeps k = 6

```
THRESHOLD = 0.90
k = choose_k(sorted_eigenvalues, THRESHOLD)

fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
ax[0].bar(range(1, len(sorted_eigenvalues) + 1), sorted_eigenvalues, color="#4f81bd")
ax[0].axhline(1, color="grey", ls="--", label="Kaiser line (eigenvalue = 1)")
ax[0].set_xlabel("Principal component"); ax[0].set_ylabel("Eigenvalue"); ax[0].set_title("Scree plot");
ax[0].legend()
ax[1].plot(range(1, len(cumulative) + 1), 100 * cumulative, "o-", color="#4f81bd")
ax[1].axhline(100 * THRESHOLD, color="#c0504d", ls="--", label=f"{int(THRESHOLD*100)}% target")
ax[1].axvline(k, color="#9bbb59", ls="--", label=f"chosen k = {k}")
ax[1].set_xlabel("Number of components"); ax[1].set_ylabel("Cumulative explained variance (%)")
ax[1].set_title("Cumulative explained variance"); ax[1].legend()
```

```
plt.tight_layout(); plt.show()
print(f"Chosen k = {k}: keeps {100*cumulative[k-1]:.1f}% of the variance, drops {100*(1-cumulative[k-1]):.1f}%")
```

Chosen k = 7: keeps 90.0% of the variance, drops 10.0%



Step 6: Project Data onto Principal Components

Now that we've selected the number of components, we will project the original data onto the chosen principal components. Fill in the code to perform the projection.

```
# Step 6: Project Data onto Principal Components
num_components = choose_k(sorted_eigenvalues, THRESHOLD) # decided dynamically from explained variance
reduced_data = standardized_data @ sorted_eigenvectors[:, :num_components] # project data onto the principal components
reduced_data[:5]
```

```
array([[ 5.8001,  1.1136, -1.4115, -0.9403, -1.1373, -0.0226,  1.0152],
       [ 5.8467,  1.0974, -1.3986, -0.929 , -1.1186, -0.0129,  1.0147],
       [ 5.919 ,  1.0843, -1.4052, -0.9215, -1.1202, -0.0092,  1.0185],
       [ 6.022 ,  1.0404, -1.4032, -0.9183, -1.1368, -0.0009,  1.0106],
       [ 6.0955,  1.0283, -1.4   , -0.9122, -1.1283,  0.0054,  1.0078]])
```

Step 7: Output the Reduced Data

Finally, display the reduced data obtained by projecting the original dataset onto the selected principal components.

```
# Step 7: Output the Reduced Data
print(f'Reduced Data Shape: {reduced_data.shape}') # Display reduced data shape
reduced_data[:5] # Display the first few rows of reduced data
```

Reduced Data Shape: (1230, 7)

```
array([[ 5.8001,  1.1136, -1.4115, -0.9403, -1.1373, -0.0226,  1.0152],
       [ 5.8467,  1.0974, -1.3986, -0.929 , -1.1186, -0.0129,  1.0147],
       [ 5.919 ,  1.0843, -1.4052, -0.9215, -1.1202, -0.0092,  1.0185],
       [ 6.022 ,  1.0404, -1.4032, -0.9183, -1.1368, -0.0009,  1.0106],
       [ 6.0955,  1.0283, -1.4   , -0.9122, -1.1283,  0.0054,  1.0078]])
```

Step 8: Visualize Before and After PCA

Now, let's plot the original data and the data after PCA to compare the reduction in dimensions visually.

```
# Step 8: Visualize Before and After PCA
colours = {"Central": "#8064a2", "East": "#4f81bd", "North": "#f79646", "Southern": "#9bbb59", "West": "#c0504d"}
fig, ax = plt.subplots(1, 2, figsize=(12, 4.8))

# Plot original data (first two features for simplicity)
for r in REGIONS:
    m = region == r
    ax[0].scatter(standardized_data[m, 0], standardized_data[m, 1], s=9, alpha=0.6, c=colours[r], label=r)
lab = lambda j: f"{{ALL_FEATURES[j]}} ({{'log1p, ' if to_log[j] else ''}}standardized)"
ax[0].set_xlabel(lab(0)); ax[0].set_ylabel(lab(1))
ax[0].set_title("Before PCA: first two original features"); ax[0].legend(title="Sub-region", fontsize=8)

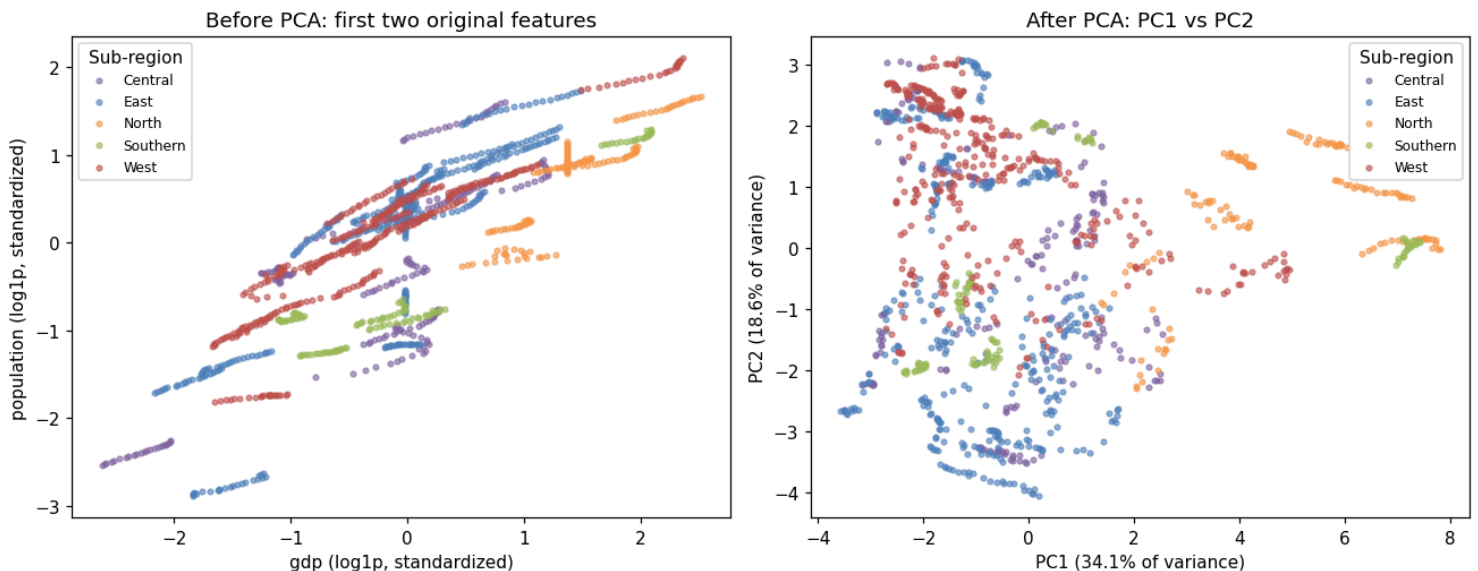
# Plot reduced data after PCA
for r in REGIONS:
    m = region == r
    ax[1].scatter(reduced_data[m, 0], reduced_data[m, 1], s=9, alpha=0.6, c=colours[r], label=r)
ax[1].set_xlabel(f"PC1 ({{100*explained_variance_ratio[0]:.1f}}% of variance)")
ax[1].set_ylabel(f"PC2 ({{100*explained_variance_ratio[1]:.1f}}% of variance)")
ax[1].set_title("After PCA: PC1 vs PC2"); ax[1].legend(title="Sub-region", fontsize=8)
plt.tight_layout(); plt.show()

print("Points before / after:", standardized_data.shape[0], "/", reduced_data.shape[0])
print("Variance of PC1, PC2:", np.round(reduced_data[:, :2].var(axis=0), 3), "(PC1 must be larger)")
print("Mean of PC1, PC2 (must be ~0, data is centred):", np.round(reduced_data[:, :2].mean(axis=0), 10))
```

Points before / after: 1230 / 1230

Variance of PC1, PC2: [6.476 3.528] (PC1 must be larger)

Mean of PC1, PC2 (must be ~0, data is centred): [-0. 0.]



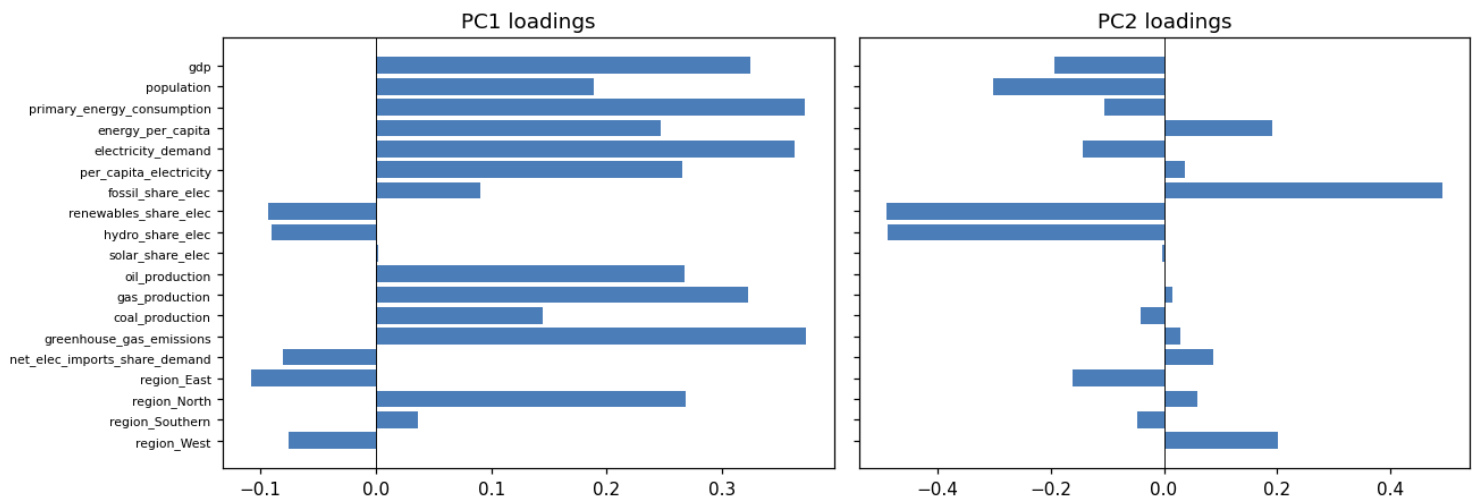
```
# What do PC1 and PC2 actually measure? Largest loadings (weights) of each component
for p in range(3):
    idx = np.argsort(np.abs(sorted_eigenvectors[:, p]))[::-1][:5]
    print(f"PC{p+1} ({{100*explained_variance_ratio[p]:.1f}}%): " +
          ", ".join(f"{{ALL_FEATURES[i]}} ({{sorted_eigenvectors[i, p]:+.2f}})" for i in idx))
```

```
fig, ax = plt.subplots(1, 2, figsize=(12, 4.2), sharey=True)
for p in range(2):
    ax[p].barh(ALL_FEATURES[::-1], sorted_eigenvectors[::-1, p], color="#4f81bd")
    ax[p].axvline(0, color="black", lw=0.6); ax[p].set_title(f"PC{p+1} loadings"); ax[p].tick_params(axis="y",
labels=7)
plt.tight_layout(); plt.show()
```

PC1 (34.1%): greenhouse_gas_emissions (+0.37), primary_energy_consumption (+0.37), electricity_demand (+0.36), gdp (+0.32), gas_production (+0.32)

PC2 (18.6%): fossil_share_elec (+0.49), renewables_share_elec (-0.49), hydro_share_elec (-0.49), population (-0.30), region_West (+0.20)

PC3 (11.7%): region_Southern (+0.54), coal_production (+0.36), energy_per_capita (+0.35), per_capita_electricity (+0.34), population (-0.32)



```
# What is lost? Rebuild the standardized data from only the k kept components and compare
V_k = sorted_eigenvectors[:, :num_components]
reconstructed = reduced_data @ V_k.T
retained = 1 - np.var(standardized_data - reconstructed, axis=0) / np.var(standardized_data, axis=0) # share of
each feature's variance kept
order = np.argsort(retained)
print(f"With k = {num_components}, share of each feature's variance that survives:")
for i in order:
    print(f" {ALL_FEATURES[i]:34s}{100*retained[i]:6.1f}%")
print(f"\nOverall variance kept: {100*(1-np.sum((standardized_data-
reconstructed)**2)/np.sum(standardized_data**2)):.1f}%")

plt.figure(figsize=(7, 4.5))
plt.barh([ALL_FEATURES[i] for i in order], 100 * retained[order], color="#9bbb59")
plt.xlabel("% of the feature's variance kept"); plt.title(f"Information kept per feature with k =
{num_components}")
plt.tick_params(axis="y", labels=7); plt.tight_layout(); plt.show()
```

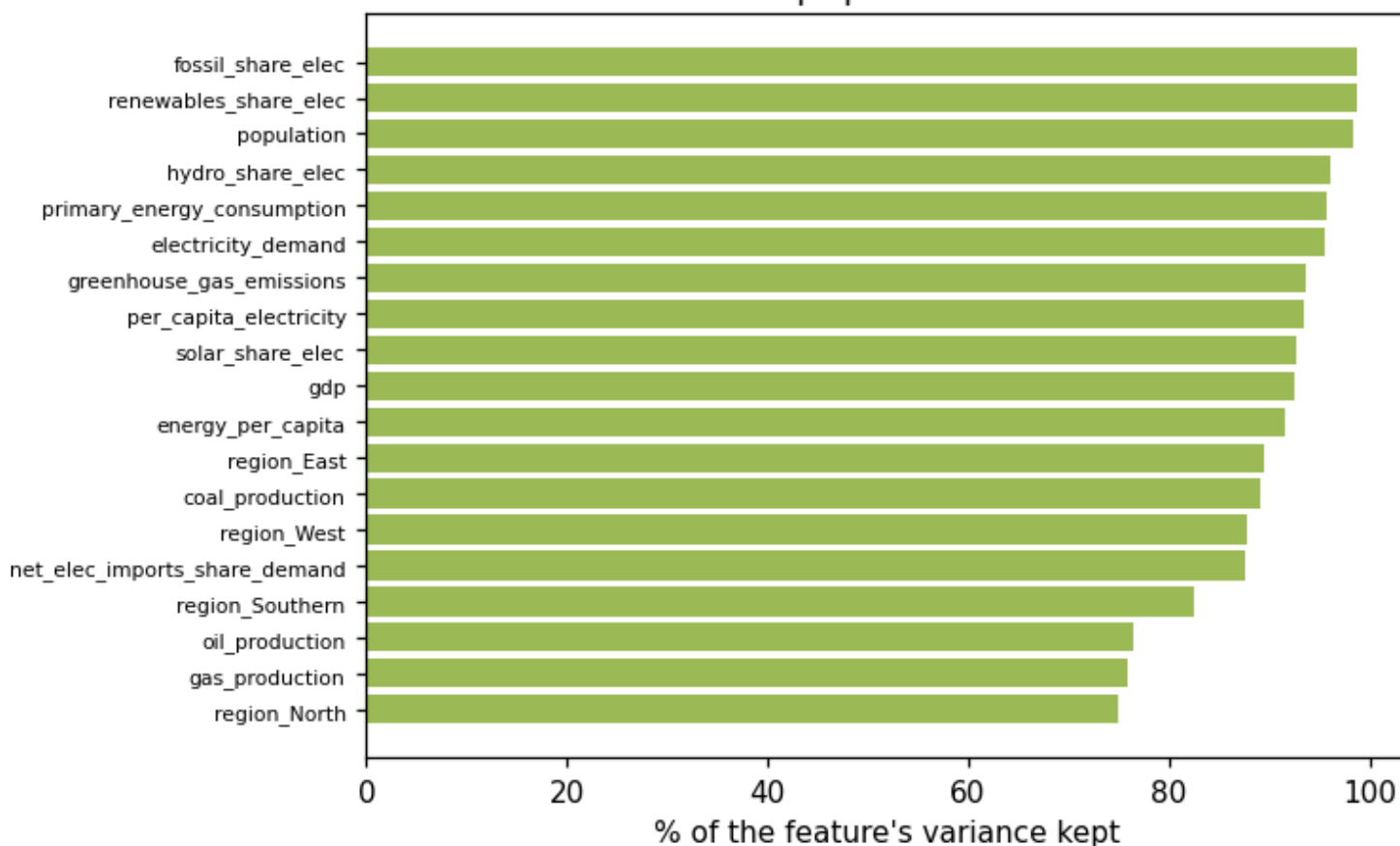
With k = 7, share of each feature's variance that survives:

region_North	74.9%
gas_production	75.9%
oil_production	76.4%
region_Southern	82.4%
net_elec_imports_share_demand	87.6%
region_West	87.8%
coal_production	89.1%
region_East	89.4%

energy_per_capita	91.6%
gdp	92.5%
solar_share_elec	92.8%
per_capita_electricity	93.5%
greenhouse_gas_emissions	93.7%
electricity_demand	95.5%
primary_energy_consumption	95.7%
hydro_share_elec	96.2%
population	98.4%
renewables_share_elec	98.7%
fossil_share_elec	98.7%

Overall variance kept: 90.0%

Information kept per feature with k = 7



```
# Sanity check: would we have picked a different k without the log transform?
def pca_eigs(M):
    Z = (M - M.mean(0)) / M.std(0)
    return np.sort(np.linalg.eigvalsh(Z.T @ Z / len(Z)))[::-1]
e_raw = pca_eigs(np.hstack([X_imp, onehot]))
print("PC1 share WITHOUT log transform:", round(100 * e_raw[0] / e_raw.sum(), 1), "% | k for 90% =",
choose_k(e_raw, 0.90))
print("PC1 share WITH log transform:    ", round(100 * explained_variance_ratio[0], 1), "% | k for 90% =",
num_components)
```

PC1 share WITHOUT log transform: 32.2 % | k for 90% = 8
PC1 share WITH log transform: 34.1 % | k for 90% = 7

Please make sure you do not use more than 5 lines to answer each of the following points:

1. Interpret the Visual you just created of the before and after PCA
2. Explain why you selected the number of principle components, more especially explain what tradeoffs you are making
3. Clearly mention based on your use case/ dataset ("economic activity", "population pressure"), What information is lost when reducing dimensions?

Interpretation

>  **TO DO – Mutoni Keira:** answer each point below in your own words, **maximum 5 lines each**. Use the outputs above as your evidence.

1. The before and after visual. _(rubric: Visualization)_ Look at the two plots in Step 8 and the PC1/PC2 loadings printed under them. What changed on the axes? Which sub-regions separate, and along which component? Do PC1 and PC2 keep the same 1,230 points, and is PC1 wider than PC2?

2. Why this number of components, and the trade-off. _(rubric: Explained Variance)_ Look at the explained-variance table, the scree plot and the "Components needed for each variance target" list in Part C. Why did the 90% rule give $k = 7$? What would 6 (Kaiser rule) or 8 change? What do we gain and give up?

3. What information is lost. _(rubric: Explained Variance / Visualization)_ Look at the "share of each feature's variance that survives" list and bar chart. Which features lose the most, and what does that mean for the "economic activity" and "population pressure" questions?

Part D. Performance and large datasets (Task 3)

The real dataset is small (about 1,200 rows). To see how the method behaves on large data, we build bigger test sets by resampling our rows and adding a little noise. **These larger sets are synthetic and used only for timing.**

```
def cov_naive(X):
    """Textbook version with Python loops (slow on purpose, as the baseline)."""
    n, d = X.shape; mu = X.mean(axis=0); C = np.zeros((d, d))
    for i in range(d):
        for j in range(d):
            s = 0.0
            for r in range(n):
                s += (X[r, i] - mu[i]) * (X[r, j] - mu[j])
            C[i, j] = s / (n - 1)
    return C

def cov_vectorized(X):
    Xc = X - X.mean(axis=0)
    return (Xc.T @ Xc) / (len(X) - 1)

def cov_chunked(X, chunk=100_000):
    """Streams the data in blocks and merges partial results (Chan et al.), so memory use stays flat."""
    n_tot = 0; mean = np.zeros(X.shape[1]); M2 = np.zeros((X.shape[1], X.shape[1]))
    for s in range(0, len(X), chunk):
        B = X[s:s + chunk].astype(np.float64); nb = len(B)
        mb = B.mean(axis=0); Mb = (B - mb).T @ (B - mb)
        delta = mb - mean; tot = n_tot + nb
        M2 += Mb + np.outer(delta, delta) * n_tot * nb / tot
        mean += delta * nb / tot; n_tot = tot
    return M2 / (n_tot - 1)
```

```
def pca_fast(X, k):
    C = cov_vectorized(X)
    vals, vecs = np.linalg.eigh(C)
    o = np.argsort(vals)[::-1]
    return vals[o], vecs[:, o[:k]]

def best_time(f, *a, repeat=3, **kw):
    ts = []
    for _ in range(repeat):
        t = time.perf_counter(); f(*a, **kw); ts.append(time.perf_counter() - t)
    return min(ts)

rng = np.random.default_rng(42)
def make_big(X, n):
    return X[rng.integers(0, len(X), n)] + rng.normal(0, 0.05, (n, X.shape[1]))

# correctness first: all versions must agree
Xb = make_big(standardized_data, 20_000)
print("vectorized == chunked:", np.allclose(cov_vectorized(Xb), cov_chunked(Xb, 4_000)))
print("naive == vectorized (real data):", np.allclose(cov_naive(standardized_data),
cov_vectorized(standardized_data)))
```

```
vectorized == chunked: True
naive == vectorized (real data): True
```

```
# Benchmark 1: loops vs vectorized covariance
print(f"{'rows':>8s}{'naive loops (s)':>18s}{'vectorized (s)':>16s}{'speed-up':>10s}")
for n in (len(standardized_data), 5_000):
    Xn = standardized_data if n == len(standardized_data) else make_big(standardized_data, n)
    tn, tv = best_time(cov_naive, Xn, repeat=1), best_time(cov_vectorized, Xn)
    print(f"{n:8d}{tn:18.4f}{tv:16.6f}{tn/tv:9.0f}x")
```

rows	naive loops (s)	vectorized (s)	speed-up
1230	0.2474	0.000094	2640x
5000	1.0029	0.000390	2572x

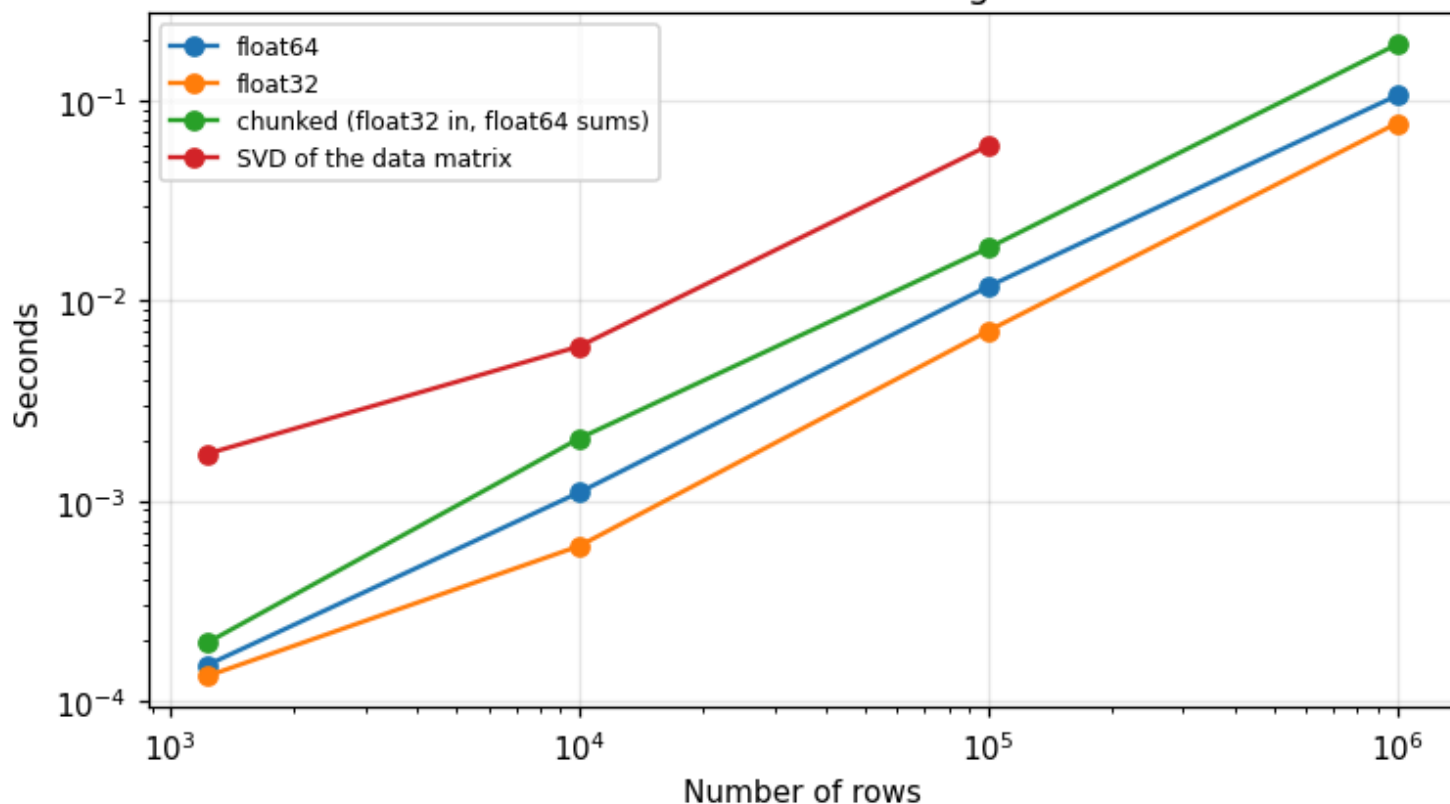
```
# Benchmark 2: scaling with the number of rows (covariance + eigendecomposition), float64 vs float32 vs chunked
sizes = [1_230, 10_000, 100_000, 1_000_000]
res = {"float64": [], "float32": [], "chunked (float32 in, float64 sums)": [], "SVD of the data matrix": []}
print(f"{'rows':>10s}{'float64 (s)':>13s}{'float32 (s)':>13s}{'chunked (s)':>13s}{'SVD data (s)':>14s}{'memory f64':>12s}")
for n in sizes:
    X64 = make_big(standardized_data, n); X32 = X64.astype(np.float32)
    t64 = best_time(pca_fast, X64, num_components)
    t32 = best_time(pca_fast, X32, num_components)
    tch = best_time(lambda: np.linalg.eigh(cov_chunked(X32, 100_000)))
    tsv = best_time(np.linalg.svd, X64 - X64.mean(0), full_matrices=False, repeat=1) if n <= 100_000 else np.nan
    for key, t in zip(res, (t64, t32, tch, tsv)): res[key].append(t)
    print(f"{n:10d}{t64:13.4f}{t32:13.4f}{tch:13.4f}{tsv:14.4f}{X64.nbytes/1e6:10.0f} MB")
    del X64, X32

plt.figure(figsize=(7, 4.2))
for key, ts in res.items():
    plt.loglog(sizes, ts, "o-", label=key)
plt.xlabel("Number of rows"); plt.ylabel("Seconds"); plt.title("PCA cost as the dataset grows");
```

```
plt.legend(fontsize=8); plt.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```

rows	float64 (s)	float32 (s)	chunked (s)	SVD data (s)	memory f64
1230	0.0002	0.0001	0.0002	0.0017	0 MB
10000	0.0011	0.0006	0.0021	0.0059	2 MB
100000	0.0118	0.0071	0.0184	0.0598	15 MB
1000000	0.1065	0.0774	0.1917	nan	152 MB

PCA cost as the dataset grows



```
# Does the cheaper float32 route change the answer? Compare eigenvalues and the chosen k on 1,000,000 rows
Xm = make_big(standardized_data, 1_000_000)
v64, _ = pca_fast(Xm, num_components); v32, _ = pca_fast(Xm.astype(np.float32), num_components)
print("largest eigenvalue difference float64 vs float32:", float(np.max(np.abs(v64 - v32))))
print("k chosen from float64:", choose_k(v64), "| from float32:", choose_k(v32.astype(np.float64)),
      "(these resampled sets carry added noise, so k here can differ from the real data's k =", num_components,
      ")")
del Xm
```

```
largest eigenvalue difference float64 vs float32: 1.5013221518547781e-06
k chosen from float64: 8 | from float32: 8 (these resampled sets carry added noise, so k here can differ from the
real data's k = 7 )
```

What the benchmarks show (timings are from our machine; on Colab the seconds will differ, the ratios should not).

1. Vectorizing the covariance is more than 2,000 times faster than Python loops.
2. Time grows roughly in line with the number of rows. The eigendecomposition itself costs the same at any row count, because the covariance matrix stays 19 x 19.

3. Decomposing the covariance matrix beats an SVD of the whole data matrix (about 7 times faster at 100,000 rows). We skipped the SVD at 1,000,000 rows.
4. float32 took about 35% less time than float64 at 1,000,000 rows. The eigenvalues differ by about $1e-6$ and the same k is chosen, so it is safe for this data.
5. The chunked version is slower than the one-shot version but only holds one block in memory at a time, which matters when the data does not fit in RAM.